Quantitative Trading: Step-by-Step Mathematical Derivations

Multi-Alpha Research Lab

This document provides the explicit, step-by-step mathematical derivations for every algorithm and statistical test used in the quantitative trading pipeline.

1 The Factor Model: Ordinary Least Squares (OLS)

To separate a strategy's pure skill (Alpha) from its market drift (Beta), the pipeline uses Ordinary Least Squares linear regression. We want to fit the equation:

$$y_t = \alpha + \beta x_t + \epsilon_t$$

Where y_t is the strategy return, x_t is the market benchmark return, and ϵ_t is the error (noise).

Step-by-Step Derivation

Step 1: Setup the Matrix Equation
We have N days of returns. We arrange our variables into matrices. Let $\mathbf{Y}$ be an $N \times 1$ vector of strategy returns. Let $\mathbf{X}$ be an $N \times 2$ matrix where the first column is all 1s (for the α intercept) and the second column is the market returns. Let $\boldsymbol{\beta}$ be a 2×1 vector containing $[\alpha, \beta]^T$.

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

Step 2: Define the Loss Function
We want to find the weights $\boldsymbol{\beta}$ that minimize the Sum of Squared Errors (SSE).

$$\text{SSE} = \boldsymbol{\epsilon}^T\boldsymbol{\epsilon} = (\mathbf{Y} - \mathbf{X}\boldsymbol{\beta})^T(\mathbf{Y} - \mathbf{X}\boldsymbol{\beta})$$

Expanding this equation using matrix algebra:

$$\text{SSE} = \mathbf{Y}^T\mathbf{Y} - 2\boldsymbol{\beta}^T\mathbf{X}^T\mathbf{Y} + \boldsymbol{\beta}^T\mathbf{X}^T\mathbf{X}\boldsymbol{\beta}$$

Step 3: Minimize via Calculus
To find the minimum, we take the derivative of the SSE with respect to the vector $\boldsymbol{\beta}$ and set it to zero:

$$\frac{\partial \text{SSE}}{\partial \boldsymbol{\beta}} = -2\mathbf{X}^T\mathbf{Y} + 2\mathbf{X}^T\mathbf{X}\boldsymbol{\beta} = 0$$

Step 4: Solve for the Weights
Divide by 2 and rearrange the equation to isolate $\boldsymbol{\beta}$:

$$\mathbf{X}^T\mathbf{X}\boldsymbol{\beta} = \mathbf{X}^T\mathbf{Y}$$

Multiply both sides by the inverse of $(\mathbf{X}^T\mathbf{X})$:

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{Y}$$

The first row of the resulting $\hat{\beta}$ matrix is the exact Alpha, and the second row is the exact Beta.

2 Orthogonality: Gram-Schmidt Decomposition

To prove that two strategies are genuinely distinct, we must mathematically subtract the redundant overlapping trades from the new strategy. This is done using Gram-Schmidt Orthogonalization.

Step-by-Step Calculation

Assume we have two strategy return vectors, $\mathbf{v}_1$ (Strategy A) and $\mathbf{v}_2$ (Strategy B).

Step 1: Define the Base Vector

The first orthogonal vector $\mathbf{u}_1$ is simply the first strategy.

$$\mathbf{u}_1 = \mathbf{v}_1$$

Step 2: Calculate the Dot Product (Overlap)

We calculate how much Strategy B moves in the exact same direction as Strategy A by multiplying their daily returns and summing them:

$$\mathbf{v}_2 \cdot \mathbf{u}_1 = \sum_{t=1}^{N} (v_{2,t} \times u_{1,t})$$

Step 3: Calculate the Magnitude of the Base

$$\mathbf{u}_1 \cdot \mathbf{u}_1 = \sum_{t=1}^{N} (u_{1,t})^2$$

Step 4: Find the Projection (The Redundant Part)

We create a scalar multiplier c which represents the exact percentage of Strategy B that is just a clone of Strategy A.

$$c = \frac{\mathbf{v}_2 \cdot \mathbf{u}_1}{\mathbf{u}_1 \cdot \mathbf{u}_1}$$

The redundant vector is therefore: $\text{Proj}_{\mathbf{u}_1}(\mathbf{v}_2) = c\mathbf{u}_1$.

Step 5: Subtract the Redundancy

We subtract the projection from Strategy B to find its pure, independent edge ($\mathbf{u}_2$).

$$\mathbf{u}_2 = \mathbf{v}_2 - \text{Proj}_{\mathbf{u}_1}(\mathbf{v}_2)$$

If $\mathbf{u}_2$ consists of all zeros, Strategy B was 100% redundant. (QR Decomposition simply applies this exact process across N strategies simultaneously in matrix form).

3 Machine Learning: Logistic Regression (Alpha05)

Alpha05 uses Logistic Regression to predict binary price direction (1 for UP, 0 for DOWN).

Step-by-Step Math

Step 1: The Linear Combination

The model receives 5 input signals (x_1 to x_5). It assigns a weight (w) to each, plus a baseline intercept (b).

$$z = (w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + w_5x_5) + b = \mathbf{w}^T\mathbf{x} + b$$

Step 2: The Sigmoid Squash
The number z could range from $-\infty$ to ∞. To convert it to a probability between 0 and 1, we use the Sigmoid function:

$$P(y = 1|\mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Step 3: Binary Cross-Entropy Loss
How does the model learn? It compares its predicted probability ($\hat{y}$) to the actual reality (y, which is either exactly 1 or 0). The probability of the model being perfectly right across all N days is the product of individual likelihoods. Because multiplying tiny probabilities causes computer underflow, we take the natural logarithm (ln), which turns multiplication into addition. This gives us the Log-Loss formula:

$$\text{Loss} = -\frac{1}{N} \sum_{i=1}^{N} [y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i)]$$

Why this works: If the truth is UP ($y_i = 1$), the right half of the bracket becomes zero. We are left with $-\ln(\hat{y}_i)$. If the model guessed 0.99, $-\ln(0.99)$ is near 0 (tiny penalty). If it guessed 0.01, $-\ln(0.01)$ is a massive positive penalty, forcing the weights **w** to update.

4 Machine Learning: Ridge Regularization & Softmax (Meta-Model)

The AlphaMetaModel operates on a tiny dataset (40 rebalances). To prevent it from memorizing noise, we add an L2 (Ridge) penalty.

Step-by-Step Math

Step 1: L2 Penalized Loss
We take the standard Log-Loss from Section 3, and add a term that squares the weights.

$$J(\mathbf{w}) = \text{Log-Loss} + \frac{1}{2C} \sum_{k=1}^{K} w_k^2$$

Step 2: The Gradient Effect
When the model calculates the derivative (gradient) to update its weights, the penalty term becomes $\frac{w_k}{C}$. This means that every single time the model updates, its weights are mathematically dragged back toward zero. The model can only assign a large weight to a feature if that feature provides an undeniably massive reduction in the Log-Loss, proving its validity.

Step 3: Softmax Allocation Conversion
The model outputs a raw conviction score s_i for each strategy. To convert these into portfolio weights that sum to 100 percent (1.0), we use Softmax. Let the scores for strategies A, B, and C be $s_A = 2.0, s_B = 0.5, s_C = -1.0$.

1. Exponentiate each score to force them to be positive: $e^{2.0} = 7.39$, $e^{0.5} = 1.65$, $e^{-1.0} = 0.37$.

2. Sum the results: $7.39 + 1.65 + 0.37 = 9.41$.

3. Divide each exponentiated score by the sum to get the final weights:

$$w_A = \frac{7.39}{9.41} = 78.5\%$$

$$w_B = \frac{1.65}{9.41} = 17.5\%$$

$$w_C = \frac{0.37}{9.41} = 4.0\%$$

5 Portfolio Optimization: Markowitz Shrinkage

Step-by-Step Math

Step 1: The Sample Covariance
We calculate the standard covariance matrix Σ_{sample} of the strategy returns. Because the sample size is small (1000 days), this matrix is mathematically unstable (nearly singular), leading to extreme allocation weights.

Step 2: The Target Matrix
We define a completely safe "target" matrix Σ_{target}. This matrix takes the variances (diagonal elements) of Σ_{sample}, but forces all cross-correlations (off-diagonal elements) to be exactly 0.

Step 3: The Shrinkage Blend
We linearly blend the unstable reality with the safe target using a shrinkage intensity λ (e.g., 0.3).

$$\Sigma_{shrunk} = (1 - \lambda)\Sigma_{sample} + \lambda\Sigma_{target}$$

Step 4: Optimal Weights
We invert the new, stable matrix and multiply it by the vector of average strategy returns $\boldsymbol{\mu}$.

$$\mathbf{w}_{raw} = \Sigma_{shrunk}^{-1}\boldsymbol{\mu}$$

We then clip negative values to 0 (no shorting) and divide by the sum of the weights to normalize to 1.0.

6 Statistical Rigor: Newey-West & Benjamini-Hochberg

Newey-West Autocorrelation Math

A standard variance calculation σ^2 assumes every day is completely independent. In trading, positions persist across days, meaning today's return is correlated with yesterday's.
Step 1: Calculate the base variance γ_0.
Step 2: Calculate the autocovariances γ_l for lags $l = 1 \dots L$.
Step 3: Apply the Bartlett kernel $(1 - \frac{l}{L+1})$, which gives less weight to older lags.
Step 4: Sum them to produce the inflated, realistic variance:

$$\hat{\sigma}^2_{NW} = \gamma_0 + 2\sum_{l=1}^{L}\left(1 - \frac{l}{L+1}\right)\gamma_l$$

By inflating the variance, the resulting t-statistic ($t = \mu/(\hat{\sigma}_{NW}/\sqrt{N})$) is smaller, making it much harder to falsely declare significance.

Benjamini-Hochberg (BH) Multiple Testing Math

If we test $m = 6$ strategies, the chance of a false positive is high. **Step 1:** Calculate the p-value for all 6 strategies and sort them from smallest to largest:

$$p_{(1)} \leq p_{(2)} \leq p_{(3)} \leq p_{(4)} \leq p_{(5)} \leq p_{(6)}$$

Step 2: Set a False Discovery Rate limit (e.g., $\alpha = 0.10$).
Step 3: Check each p-value against a dynamically scaling threshold: $\frac{k}{m}\alpha$, where k is the rank.

- For the best strategy ($k = 1$), it must beat $\frac{1}{6} \times 0.10 = 0.016$.
- For the second best ($k = 2$), it must beat $\frac{2}{6} \times 0.10 = 0.033$.

If the best strategy achieved a p-value of 0.04, it fails the test (0.04 is not less than 0.016). This mathematically punishes researchers for throwing multiple random ideas at the wall to see what sticks.